

# CIBERGUARDIÁN

Asistente Ciudadano de Auxilio Rápido, Prevención y Detección de Brotes ante Estafas Digitales

Especificación Técnica Integral de Módulos de Respuesta, Análisis de Amenazas, y Herramientas de Monitoreo, con un Catálogo Exhaustivo de Librerías y Paquetes de Software, y un Análisis de Vulnerabilidades y Momentos Críticos del Ciberdelito.

## EJE TEMÁTICO

Seguridad y Sociedad

## DESAFÍO ASIGNADO

Detección de Riesgos y Fraudes Digitales

## METODOLOGÍA

Spec-Driven Development (SDD) + TDD

## VERSIÓN & FECHA

1.0 Oficial Consolidada • Septiembre 2026

# 1 Resumen Ejecutivo y Paradigma de Solución

## 1.1 Diagnóstico de la Problemática

Las ciberestafas modernas (phishing bancario, robos de cuenta de WhatsApp, llamadas con secuestro emocional, sitios clonados y desvío de transferencias) **no reconocen fronteras barriales ni límites físicos**. Su característica diferencial radica en la **manipulación psicológica**: los estafadores inducen urgencia artificial, miedo a la pérdida o suplantación de autoridad pública o financiera, vulnerando la toma de decisiones racional en cuestión de segundos.

### ⚠ La Brecha Crítica en la Seguridad Ciudadana

La seguridad ciudadana no se resuelve con un antivirus técnico instalado en los servidores, porque los atacantes no vulneran código: manipulan a las personas. La brecha ocurre en tres tiempos: **(1) Antes**: ausencia de un canal de consulta instantáneo (<5s) cuando surge la duda; **(2) Durante**: falta de una guía que interrumpa el secuestro emocional durante una llamada en vivo; y **(3) Después**: desamparo absoluto en los 15 minutos posteriores al engaño, sin acceso inmediato a líneas 24hs ni preservación formal de la evidencia digital.

## 1.2 Tesis del Proyecto: "Chatbot-First & Ecosistema Multi-Canal"

CiberGuardián elimina toda fricción de registro o descargas pesadas mediante una estrategia de dos puntas contextualizada en el uso cotidiano de la tecnología:

### 💻 En Computadora (PC)

Extensión Web Manifest V3

Centinela pasivo en segundo plano. Provee *menú contextual* con clic derecho en **WhatsApp Web** o **Gmail** ("Analizar con CiberGuardián"), *detector de formularios sospechosos* que bloquea campos de tokens/claves en dominios apócrifos antes de enviar datos, y un panel lateral interactivo completo.

### 📱 En Smartphone (Celular)

PWA + Web Share Target

Cero fricción de descarga. La PWA se ancla a la pantalla de inicio. Al recibir un mensaje o SMS dudoso en WhatsApp móvil: *mantener presionado* → *Compartir* → *CiberGuardián*. El bot se abre precargado y devuelve el veredicto en menos de 3 segundos con privacidad absoluta (no espía conversaciones).

## 1.3 Inclusión y Accesibilidad Universal

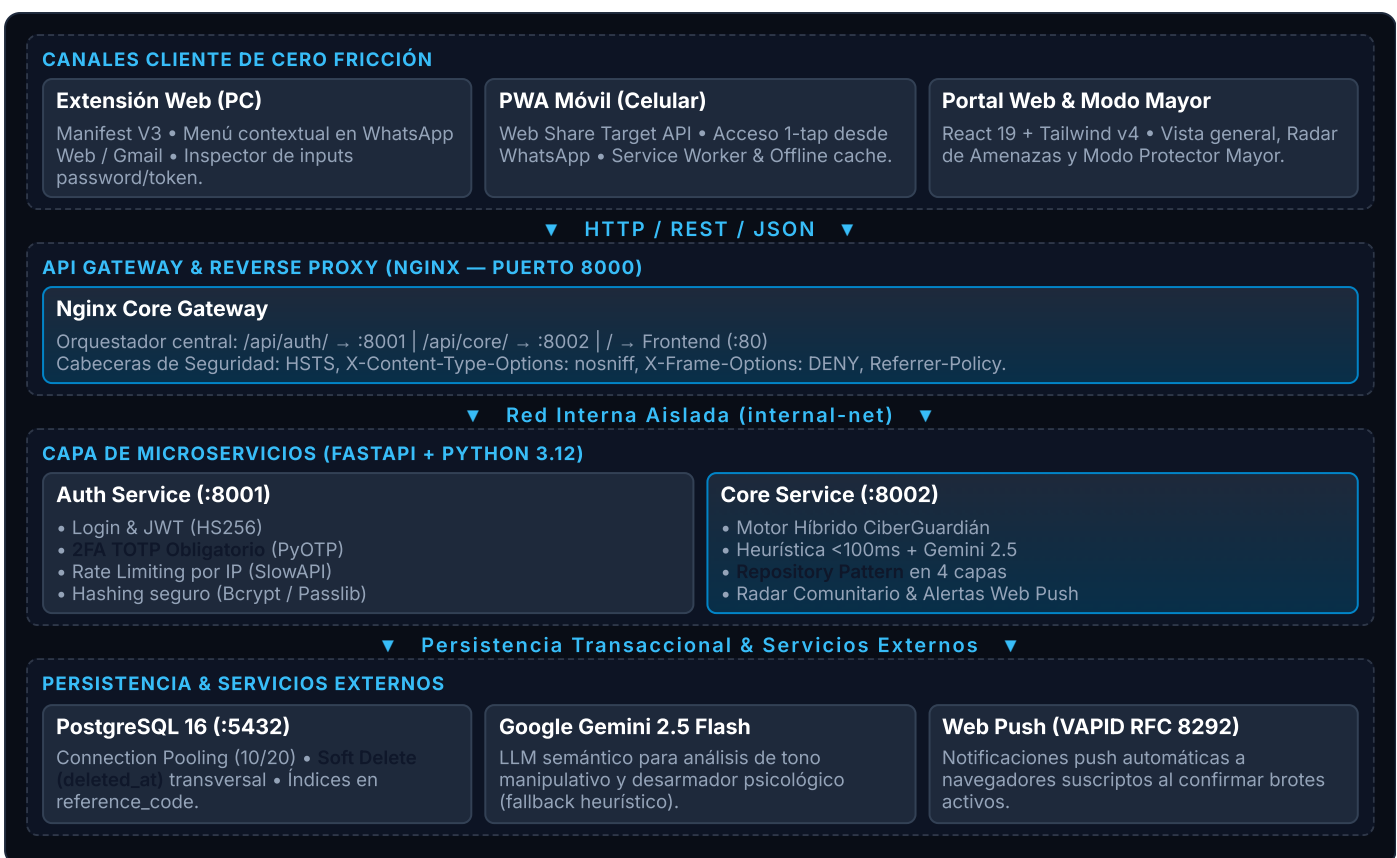
El sistema incorpora de manera nativa el **Modo Protector Mayor**, una interfaz pensada para adultos mayores con tipografía extra grande, botones de gran tamaño, contraste visual reforzado y eliminación total de tecnicismos en inglés (*phishing* pasa a ser "*sitio falso*" y *malware* pasa a ser "*programa espía*").

### 🎯 Objetivo Estratégico

Democratizar la ciberseguridad mediante una herramienta que no exige conocimientos técnicos, vive en los dispositivos donde ocurren los engaños y articula prevención temprana con auxilio legal y bancario inmediato.

## 2 Arquitectura General del Sistema

CiberGuardián adopta una arquitectura de microservicios containerizada con Docker Compose, orquestada mediante un API Gateway Nginx que centraliza el enrutamiento, la terminación segura y las políticas de CORS.



## 2.1 Principios Arquitectónicos Aplicados

 **Repository Pattern en Backend**

En el microservicio Core, se implementa una separación estricta: **Router** → **Service** → **Repository** → **Database**. La lógica de negocio reside exclusivamente en los servicios, garantizando desacoplamiento total, testabilidad unitaria y cero dependencias directas en routers.

 Borrado Lógico (Soft Delete) deleted\_at

Todas las entidades relacionales (usuarios, reportes de incidentes, votos, historial de chat) implementan la columna `deleted_at`. Jamás se eliminan físicamente registros, preservando la cadena de custodia de evidencia digital para peritajes judiciales y auditorías.

### 3 Stack Tecnológico y Seguridad Criptográfica

La selección tecnológica prioriza el alto rendimiento asincrónico, la tipificación estricta de extremo a extremo y el cumplimiento riguroso de las directivas técnicas de la competencia.

-  **Backend: Python 3.12 + FastAPI** Microservicios
  - **FastAPI (v0.115+)**: Framework web de alto rendimiento basado en Starlette y Pydantic v2, de naturaleza asincrónica (ASGI), con contratos OpenAPI / Swagger automáticos.
  - **SQLAlchemy 2.0**: ORM industrial con consultas modernas 2.0 ( `select()` ), mapeo declarativo tipado y pooling de conexiones optimizado (10 activas, 20 overflow).
  - **PostgreSQL 16 Alpine**: Motor relacional de máxima robustez transaccional (ACID), con índices optimizados y soporte de filtrado eficiente sobre borrado lógico.
  - **Nginx**: Reverse Proxy de ultra-baja latencia actuando como API Gateway y unificador de CORS y cabeceras de blindaje HTTP.
-  **Frontend: React 19 + TypeScript** Single Page App & PWA
  - **React 19 (v19.2+)**: Última versión estable con compilador optimizado, transiciones fluidas y renderizado concurrente sin bloqueos de interfaz.
  - **TypeScript 6.0**: Tipado estático riguroso en toda la aplicación cliente, garantizando compatibilidad contractual con los schemas de Pydantic.
  - **Vite 8.3**: Bundler de última generación con Hot Module Replacement (HMR) instantáneo y optimización de paquetes para producción.
  - **Tailwind CSS v4**: Framework de utilidades CSS moderno con motor Rust ( `@tailwindcss/vite` ), proporcionando diseño responsivo y Modo Protector Mayor.

-  **Extensión Web: Manifest V3** Desktop Client
  - **Chrome Extension Manifest V3**: Cumplimiento del estándar más moderno y seguro para extensiones en navegadores Chromium (Chrome, Brave, Edge).
  - **Background Service Worker**: Monitoreo no invasivo y gestión de eventos del menú contextual del sistema sin penalización de memoria.
  - **Content Script Heurístico**: Detección pasiva en tiempo real de campos de entrada sensibles ( `password` , `token` , `cbu` ) en dominios no bancarios.
-  **Inteligencia Artificial Híbrida** Motor Dual
  - **Motor Heurístico Local (<100ms)**: Algoritmos de expresiones regulares y análisis de patrones léxicos para detección ultrarrápida de urgencia, dominios falsos y pedidos de token.
  - **Google Gemini 2.5 Flash**: Integrado mediante el SDK oficial `google-genai` para comprensión semántica profunda, categorización de intencionalidad y desarmador psicológico.
  - **Estrategia Fallback**: En caso de microcortes o latencia de red, el motor heurístico local garantiza servicio y diagnóstico ininterrumpido.

#### 3.1 Arquitectura de Seguridad y Criptografía

-  **Segundo Factor 2FA TOTP Mandatorio** PyOTP + RFC 6238

Implementación rigurosa del estándar TOTP mediante `pyotp` . Emisión de secretos criptográficos seguros y generación de códigos QR en Base64 con `qrcode[pil]` para sincronización con Google Authenticator y Authy. Obligatorio para moderadores.
-  **Rate Limiting y Blindaje de Red** SlowAPI + Security Headers

Control estricto de tasa de solicitudes por dirección IP mediante `slowapi` en endpoints de login, registro y chat. Cabeceras HTTP de blindaje en Gateway: HSTS, X-Content-Type-Options: nosniff, X-Frame-Options: DENY y Referrer-Policy restrictiva.
-  **Protección de Credenciales y Sesiones**

Contraseñas hashadas exclusivamente mediante el algoritmo **Bcrypt** a través de `passlib` con salting dinámico y factor de trabajo seguro. Sesiones emitidas mediante **JSON Web Tokens (JWT)** firmados simétricamente con algoritmo `HS256` y validación estricta de tiempo de expiración y roles ( `moderator` , `admin` ).

## 4 Matriz Detallada de Librerías y Dependencias

Ecosistema de dependencias de producción y testing de cada servicio, con versiones fijadas y justificación técnica:

### 4.1 Backend: Auth Service (:8001) y Core Service (:8002)

| Librería / Paquete                       | Versión        | Módulo      | Propósito y Justificación Técnica                                                                                   |
|------------------------------------------|----------------|-------------|---------------------------------------------------------------------------------------------------------------------|
| <code>fastapi</code>                     | ≥ 0.115.0      | Auth / Core | Framework web asincrónico (ASGI), inyección de dependencias, tipado con Pydantic y Swagger nativo.                  |
| <code>uvicorn[standard]</code>           | ≥ 0.32.0       | Auth / Core | Servidor ASGI de alto rendimiento con optimizaciones de uvloop y httptools para concurrencia masiva.                |
| <code>pydantic</code>                    | ≥ 2.9.0        | Auth / Core | Validación estricta de entrada y salida con tipado de Pydantic V2, validadores de campo y serialización.            |
| <code>pydantic-settings</code>           | ≥ 2.5.0        | Auth / Core | Gestión declarativa y validada de variables de entorno ( <code>.env</code> ) y configuración inmutable del sistema. |
| <code>sqlalchemy</code>                  | ≥ 2.0.35       | Auth / Core | ORM relacional Data Mapper con sintaxis 2.0 ( <code>select()</code> ), transacciones ACID y pooling de conexiones.  |
| <code>psycopg2-binary</code>             | ≥ 2.9.9        | Auth / Core | Driver PostgreSQL compilado en C para operaciones sincrónicas de alta velocidad con SQLAlchemy.                     |
| <code>psycopg[binary]</code>             | ≥ 3.2.0        | Core        | Driver PostgreSQL de tercera generación con mejoras en pipelining, concurrencia y soporte tipado.                   |
| <code>pyotp</code>                       | ≥ 2.9.0        | Auth        | Generación y verificación de códigos de un solo uso por tiempo (TOTP RFC 6238) para autenticación 2FA.              |
| <code>qrcode[pil]</code>                 | ≥ 7.4.2        | Auth        | Generación dinámica de imágenes QR en memoria (Base64) para enrolar cuentas en Google Authenticator / Authy.        |
| <code>passlib[bcrypt]</code>             | ≥ 1.7.4        | Auth        | Criptografía y salting seguro de contraseñas con el estándar Bcrypt para mitigar ataques de fuerza bruta.           |
| <code>python-jose[crypto]</code>         | ≥ 3.3.0        | Auth / Core | Firma simétrica y verificación criptográfica de tokens JWT (HS256) para autorización segura.                        |
| <code>slowapi</code>                     | ≥ 0.1.9        | Auth / Core | Middleware de limitación de tasa (Rate Limiting) basado en límites por dirección IP para prevenir abusos.           |
| <code>google-genai</code>                | ≥ 2.0.0        | Core        | SDK oficial de Google para inferencia con Gemini 2.5 Flash, análisis semántico y desarmador psicológico.            |
| <code>pywebpush</code>                   | ≥ 2.0.0        | Core        | Envío de notificaciones push cifradas según la especificación VAPID (RFC 8292) a navegadores web.                   |
| <code>pytest</code> / <code>httpx</code> | ≥ 8.3.0 / 0.27 | Auth / Core | Suite de testing automatizado y cliente HTTP asincrónico para pruebas unitarias y de integración TDD.               |

### 4.2 Frontend (React 19 + TypeScript) y Extensión Web

| Librería / Paquete                              | Versión          | Área              | Propósito y Justificación Técnica                                                                                 |
|-------------------------------------------------|------------------|-------------------|-------------------------------------------------------------------------------------------------------------------|
| <code>react</code> / <code>react-dom</code>     | ^19.2.8          | Frontend UI       | Librería nuclear de construcción de interfaces con nuevo compilador y hooks reactivos concurrentes.               |
| <code>typescript</code>                         | ~6.0.2           | Frontend / Ext    | Garantiza tipado estático, autocompletado y ausencia de errores en tiempo de compilación.                         |
| <code>vite</code>                               | ^8.3.0           | Tooling           | Servidor de desarrollo ultrarrápido y empaquetador para producción optimizado mediante Rollup.                    |
| <code>tailwindcss</code>                        | ^4.3.3           | Estilos UI        | Framework de utilidades CSS de cuarta generación con motor Oxide ultrarrápido ( <code>@tailwindcss/vite</code> ). |
| <code>lucide-react</code>                       | ^1.47.0          | Iconografía       | Iconos vectoriales limpios y accesibles. <b>Cumple la regla estricta de cero emojis como iconos.</b>              |
| <code>sonner</code>                             | ^2.0.8           | Feedback / Toasts | Sistema de notificaciones toast moderno y accesible. <b>Reemplazo total y estricto de alert().</b>                |
| <code>axios</code>                              | ^1.20.0          | Cliente HTTP      | Comunicación con el API Gateway, interceptores de autenticación y manejo de errores estandarizado.                |
| <code>clsx</code> / <code>tailwind-merge</code> | ^2.1.1 / ^3.7.0  | Utilidades UI     | Fusión condicional y sin conflictos de clases de Tailwind CSS en componentes dinámicos y estados.                 |
| <code>vitest</code> / <code>testing-lib</code>  | ^5.0.1 / ^16.3.3 | Testing Frontend  | Ejecución de pruebas unitarias y de integración de componentes React con DOM virtual y aserciones Jest.           |
| <code>axe-core</code>                           | ^4.13.0          | Accesibilidad     | Validación automatizada de estándares WCAG de accesibilidad web (crucial en Modo Protector Mayor).                |

## 5 Funcionamiento General del Sistema

El sistema se estructura en torno al **ciclo integral de respuesta ante el ciberdelito**, resolviendo de forma diferenciada cada uno de los 3 momentos críticos de la estafa:

MOMENTO 1 • PREVENCIÓN

**Antes de Dudar (<100ms)**  
"Desarmá la trampa antes de hacer clic"  
Análisis instantáneo de mensajes dudosos, SMS o enlaces copiados o compartidos desde WhatsApp.

- Semáforo visual (🟢 Seguro / 🟡 Precaución / 🔴 Peligro) y Score (0-100%).
- Desarmador Psicológico:** Desmenuza la urgencia artificial y la suplantación de identidad.
- Botón 1-Tap a Familiar:** Abre WhatsApp ( [wa.me](#) ) con el análisis redactado para consultar a un allegado.

MOMENTO 2 • CONTENCIÓN

**Durante la Llamada (En Vivo)**  
"Me están llamando o apurando ahora mismo"  
Botón de pánico que corta el secuestro emocional de la víctima en tiempo real.

- Mensaje visual imperativo en rojo: *"¡CORTÁ LA LLAMADA YA! Ningún banco te pide ir al cajero".*
- Guía rápida de 3 preguntas de descarte en 10 segundos.
- Intervención de emergencia sin necesidad de leer textos extensos ni memorizar protocolos.

MOMENTO 3 • AUXILIO SOS

**Después del Incidente (SOS)**  
"¡Pasé mis datos o transferí plata, auxilio!"  
Protocolo de primeros auxilios digitales en los 15 minutos de oro post-estafa.

- Botones 1-Tap a Líneas 24hs:** Bloqueo directo de Red Link, Banelco y Banco Formosa.
- Ficha de Denuncia Digital:** Genera documento estructurado con CBU y código pericial.
- Inmunidad Colectiva:** Alimenta anónimamente el Radar Comunitario de brotes.

### 5.1 Flujos Operativos Detallados

 **Flujo 1: Análisis Heurístico y Desarmador de Manipulación Psicológica**

POST /api/core/chat/message

El usuario ingresa un texto o enlace. En menos de 100ms, el motor heurístico local evalúa patrones léxicos críticos (bloqueos inmediatos de cuenta, solicitud de tokens, dominios falsos como [banco-redlink.xyz](#)). Si el caso requiere interpretación de lenguaje ambiguo, Google Gemini 2.5 Flash clasifica la técnica de persuasión coercitiva y redacta una explicación desmitificadora. La respuesta incluye semáforo, score de riesgo, fragmentos señalados con su justificativo y sugerencias de acción directa.

 **Flujo 2: Radar Comunitario de Amenazas y Detección de Brotes**

GET /api/core/incidents

El radar cataloga incidentes por **Entidad Suplantada** (Banco Formosa, Mercado Pago, Correo Argentino, etc.) y **Vector de Ataque** (WhatsApp, SMS, Llamada, Web clonada). Opera con paginación server-side ( [page](#) , [limit](#) ) y filtros dinámicos. Los usuarios pueden registrar coincidencias con el botón *"A mí también me llegó"*. Al superarse el umbral de incidencia en una ventana temporal, el sistema dispara automáticamente una **Alerta de Brote Activo** y despacha notificaciones Web Push cifradas.

 **Modo Protector Mayor**

Accesibilidad Universal

Pensado para adultos mayores: interfaz con tipografías extra grandes (20pt+), contraste reforzado, botones táctiles gigantes y eliminación total de tecnicismos en inglés (*phishing* pasa a ser *"sitio tramposo"*; *malware* pasa a ser *"programa espía"*).

 **Panel de Moderación Institucional**

2FA TOTP con Google Authenticator

Las autoridades y moderadores acceden mediante login seguro validado por contraseña Bcrypt y código dinámico de 6 dígitos emitido por Google Authenticator / Authy. Tienen facultades para auditar denuncias, confirmar brotes y aplicar [deleted\\_at](#) (Soft Delete) a reportes apócrifos.

## 6 Cumplimiento Riguroso de la Guía Técnica del IPF

El proyecto ha sido diseñado para acatar con máxima fidelidad cada directiva técnica y de calidad exigida por el jurado evaluador de FormosaHack:

| Exigencia Técnica de la Guía     | Estándar Implementado                                 | Validación Técnica en el Repositorio                                                                                                                                                        |
|----------------------------------|-------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Cero Alertas Nativas (alert())   | Sonner Toasts + Modales Accesibles                    | 100% notificaciones con <code>toast.success()</code> , <code>toast.error()</code> y componente <code>ConfirmModal</code> . Cero llamadas a <code>window.alert()</code> en todo el frontend. |
| Arquitectura de Microservicios   | Nginx Gateway + FastAPI Microservices                 | Gateway en puerto 8000 enrutando de forma aislada a <code>auth_service</code> (8001), <code>core_service</code> (8002) y <code>frontend</code> (80).                                        |
| Repository Pattern en Backend    | Separación en 4 Capas                                 | Microservicio Core desacoplado en: <code>Router</code> → <code>Service</code> → <code>Repository</code> → <code>Database</code> con SQLAlchemy 2.0.                                         |
| Borrado Lógico (Soft Delete)     | Columna <code>deleted_at</code> transversal           | Ninguna entidad se purga con <code>DELETE</code> . Se preserva la evidencia digital con fecha de eliminación lógica para peritajes.                                                         |
| Paginación y Filtros en Servidor | Paginación con <code>page</code> y <code>limit</code> | Endpoints de incidentes y recursos con paginación server-side y filtros dinámicos por entidad, vector y término de búsqueda.                                                                |
| Seguridad 2FA TOTP Obligatoria   | RFC 6238 con PyOTP                                    | Autenticación de dos factores mandatoria con Google Authenticator / Authy para moderadores institucionales.                                                                                 |
| Rate Limiting Estricto           | SlowAPI en Endpoints Críticos                         | Límites de peticiones por dirección IP en endpoints de login, registro e interacción con el motor de chat.                                                                                  |
| Iconografía Profesional          | Lucide React Exclusivo                                | Prohibición total de emojis de texto en componentes UI; empleo riguroso de SVG vectoriales optimizados.                                                                                     |

✓ Conclusión del Sistema

CiberGuardián transforma una arquitectura técnica de alta exigencia (microservicios FastAPI, 2FA TOTP, Repository Pattern, PostgreSQL 16 y React 19) en una **herramienta ciudadana de impacto social directo y cero fricción**. Al acompañar al usuario antes de dudar, durante la llamada y después del incidente, el sistema ofrece una respuesta real, inclusiva y definitiva al flagelo de las ciberestafas en la provincia y el país.